Servizi e funzioni del Sistema Operativo

Introduzione

Dopo aver visto come si compone un sistema operativo andiamo a vedere quali sono le sue funzioni e servizi.

Gestione dei processi

Innanzitutto un Sistema Operativo si occupa della gestione dei processi all'interno della macchina a livello di software, ma cos'è un **processo?**

> **Un processo è un istanza di un applicazione o di un programma in esecuzione.**

Ad esempio, in un computer o dispositivo con un Multi-tasking Operating System (OS), più processi vengono eseguiti simultaneamente, sia che appartengano ad applicazioni diverse sia alla stessa applicazione.
Quindi il S.O. gestisce questi processi attraverso varie attività:

- **Creazione e terminazione dei processi**.
- **Sospensione e ripresa dei processi in esecuzione.**
- **Sincronizzazione e facilitazione delle comunicazioni tra i processi.<**
- Gestione delle situazioni di **deadlocks:**
  - **sono situazioni in cui due o più processi restano in attesa l'uno dell'altro, bloccandosi a vicenda.**
  I processi possono fare **chiamate di sistema** per interagire con il Sistema Operativo e gestire la loro esecuzione.
  Alcuni esempi includono:
- `exec`:
  Eseguire un nuovo processo sostituendo quello attuale.
- **`fork`:**
  Creare un nuovo processo duplicando quello in esecuzione.
- `wait/signal`:
  Sincronizzare e scambiare segnali tra processi.
- `kill/terminate`:
  Terminare un processo in esecuzione

Fork

La chiamata di sistema `fork` consente a un processo di creare una copia di se stesso, generando così un nuovo processo figlio.
Questo meccanismo porta alla formazione di una **gerarchia padre-figlio** tra i processi in esecuzione:

- **Il processo padre è il processo originale che richiama `fork`.**
- **Il processo figlio è la copia del padre, con lo stesso stato e le stesse risorse iniziali.**

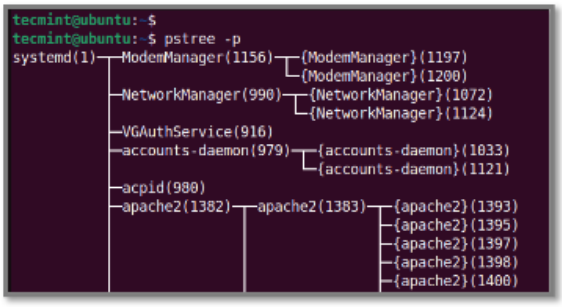
Questa struttura porta alla creazione di un **albero di processi**, in cui il primo processo avviato dal sistema operativo (nelle moderne distribuzioni Linux noto come `systemd` o, nei sistemi UNIX meno recenti, come `init`) funge da radice e genera tutti gli altri processi del sistema.

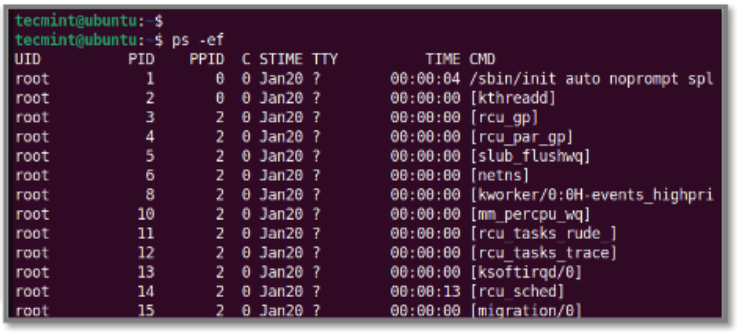
Identificazione dei processi

Ogni processo in esecuzione è identificato da tre parametri fondamentali:

- **Process ID (PID):**
  Identificatore univoco del processo.
- **Parent Process ID (PPID):**
  Indica il PID del processo padre.
- **User ID (UID):**
  Specifica l'utente che ha avviato il processo.

Questi identificatori permettono al sistema operativo di gestire le relazioni tra i processi e di controllarne l'accesso alle risorse.





center

Come funziona il fork ›

Quando un processo esegue la chiamata di sistema `fork`, il SO crea una copia quasi identica del processo chiamante.
Il processo originale viene chiamato **processo padre**, mentre la nuova copia viene chiamata **processo figlio**.
Entrambi i processi continuano l'esecuzione dallo stesso punto subito dopo la chiamata `fork`, ma con **identificatori di processo (PID) diversi**.
L'unica differenza immediata tra i due è il valore restituito da `fork`:

- Al **processo padre**, `fork` restituisce il **PID del figlio**
- Al processo figlio, `fork` restituisce **0.**
  Questa distinzione permette ai due processi di eseguire operazioni diverse in base al loro ruolo.

Esempio

Un processo padre potrebbe attendere la terminazione del figlio con `wait()`, mentre il figlio potrebbe eseguire una nuova applicazione con `exec()`.

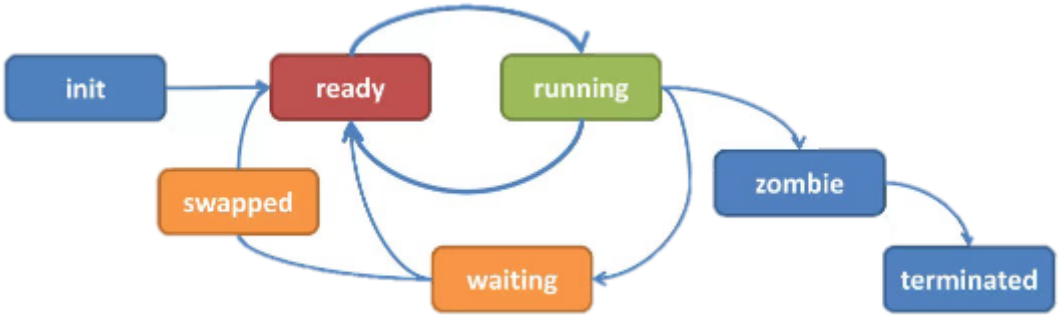
Il ciclo di vita di un processo

Ogni processo ha la sua durata di vita, che prevede 7 stati del ciclo di vita:

1. **Init:**
   - **È lo stato iniziale del processo in memoria.**
   - **Il processo viene inizializzato e reso eseguibile.**
   - Quindi in questo stato viene creato il processo,
     e il SO alloca la memoria RAMnecessaria per l'esecuzione.
2. **Ready:**
   - **Il processo viene caricato RAM ed è pronto per essere eseguito dal processore.**
3. **Running:**
   - **il processo è in esecuzione sulla CPU.**
4. **Waiting:**
   - **il processo viene sospeso in RAM in attesa che si verifichi un evento (es: I/O)**.

5. **Swapped:**
   - **il processo mentre aspetta l'evento viene messo in memoria virtuale (swap); si trova sul disco rigido (Hard Disk), ma viene usato come se fosse una RAM**
6. **Zombie:**
   - **Il processo entra in questo stato quando ha terminato l'esecuzione ma il processo padre non ha ancora recuperato il suo stato con `wait()`**:
     - **di conseguenza il processo figlio aspetta di essere terminato dal processo padre**
7. **Terminated:**
   - **il processo viene terminato e la memoria allocata viene liberata, in parole povere il processo padre ha "killato" il processo figlio**.

Lo stato del processo non cambia a caso:



Process Life Cycle.png

Osservando questa immagine possiamo comprendere la dinamicità del ciclo di vita dei processi; mostra come un processo possa cambiare stato a seconda delle condizioni di esecuzione e della gestione delle risorse nel sistema operativo.

Spiegazione dell'immagine sugli stati del ciclo di vita.

1. **Init → Ready:**

- **Quando un processo viene creato, il sistema operativo lo alloca in memoria (RAM) e lo prepara per l'esecuzione.**
- A questo punto, il processo si trova nello stato **Ready:**
  - **in attesa di essere eseguito dal processore**.

2. **Ready → Running:**

- **Se la CPU è disponibile, il processo viene schedulato per l'esecuzione e passa allo stato Running.**

3. **Running → Waiting:**

- **Se il processo necessita di attendere un evento esterno (ad esempio, l'input da tastiera o la lettura di un file), viene sospeso e messo nello stato di **Waiting**.**
- **Rimane in Waiting fino a quando l'evento si verifica.**

4. **Waiting → Ready:**

- **Quando l'evento atteso si verifica (ad esempio, un file viene caricato in memoria o arriva un input dall'utente), il processo torna nello stato Ready, pronto per essere nuovamente eseguito.**

5. **Running → Swapped:**

- **Se il sistema operativo decide di liberare memoria RAM, può spostare un processo inattivo nella memoria virtuale (swap).**
- Il processo entra nello stato **Swapped:**
  - **restando in attesa di essere riportato in RAM.**
  6)**Swapped → Ready:**
  - **Se il processo deve essere rieseguito, viene ricaricato in RAM e torna nello stato Ready.**

7. **Running → Zombie:**
   - **Quando un processo completa la sua esecuzione ma il processo padre non ha ancora raccolto il suo stato con `wait()`, entra nello stato Zombie.**
8. **Zombie → Terminated:**
   Una volta che il processo padre esegue `wait()`:
   - **il processo figlio viene rimosso completamente e passa allo stato Terminated, liberando le risorse allocate.**

Osservazioni sulla struttura dell'immagine

- Le **frecce** mostrano chiaramente che un processo può passare **da Ready a Running**, ma può anche tornare in Ready dopo Waiting o Swapped.

- La transizione tra **Running e Zombie** avviene solo quando il processo ha finito l'esecuzione ma non è stato ancora eliminato.
- Non esiste un collegamento diretto tra **Zombie e altri stati** (tranne Terminated), perché una volta che un processo diventa Zombie, l'unico modo per eliminarlo è tramite il padre.

Gestione memoria centrale

La memoria centrale include i registri della CPU (IR, PC, ACC, etc.), la cache della CPU e la RAM.
Il SO per quanto riguarda la gestione della memoria principale ha diversi responsabilità:

- **Allocazione e de-allocazione della memoria**:
  Il SO assegna memoria ai processi quando necessario e la libera quando il processo termina.
- **Isolamento dei segmenti di memoria**:
  Nei sistemi multi-tasking**, il SO garantisce che i diversi processi abbiano spazi di memoria separati per prevenire conflitti e accessi non autorizzati.**
- **Mapping degli indirizzi**:
  Il SO gestisce la traduzione tra gli indirizzi logici (usati dai processi) e gli indirizzi fisici (posizioni reali nella RAM).
- **Gestione della memoria virtuale**:
  Il SO utilizza la tecnica dello swap per spostare porzioni di un programma tra la memoria primaria (RAM) e la memoria secondaria (disco rigido) per ottimizzare l'uso delle risorse.

Addressing Mapping

Ogni volta che un processo viene eseguito, gli viene assegnato un **spazio di indirizzi virtuale:**

> **che appare come un'area continua e coerente a partire da zero**.

Tuttavia, questo spazio di memoria virtuale non è necessariamente contiguo nella memoria fisica (RAM).

Il sistema operativo, con l'aiuto della **Memory Management Unit (MMU)**, si occupa di **mappare** gli indirizzi virtuali a quelli fisici.
Questo significa che quando un processo accede a un indirizzo virtuale, in realtà il sistema operativo lo traduce in un indirizzo fisico in RAM.
Quindi questo permette al SO di:

- **Allocare memoria in modo contiguo.**
- **Proteggere le aree di memoria tra i processi**.
- **Implementare la memoria virtuale.**

> **Ad Esempio:**
>
> Se un processo accede all'indirizzo virtuale `0x0040A0B0`, la MMU lo traduce in un indirizzo fisico reale, ad esempio `0x12F3C0B0`, consultando la tabella delle pagine.

Caratteristiche principali della gestione della memoria

- **Isolamento**:
  - **Ogni processo (come un'applicazione o un servizio di background) ha il proprio spazio di indirizzi virtuale, un insieme di indirizzi di memoria che può utilizzare per accedere alle risorse**.
  - **Questo spazio è privato e non accessibile dagli altri processi, garantendo protezione e impedendo interferenze, il che migliora sicurezza e stabilità**.
- **Coerenza**:
  - Per il processo, lo **spazio di memoria virtuale** appare come un **intervallo continuo di indirizzi** a partire da zero, **indipendentemente da come la memoria fisica (RAM) è strutturata o da quanta memoria sia effettivamente disponibile.**
  - **Il sistema operativo si occupa di gestire questa astrazione**.
- **Mappatura**:
  - Gli indirizzi virtuali utilizzati dal processo non corrispondono direttamente a quelli fisici.
  - **Quando un processo accede a un indirizzo di memoria, in realtà sta accedendo a un indirizzo virtuale.**
  - **Il sistema operativo, con l'aiuto della Memory Management Unit (MMU), traduce questi indirizzi virtuali in indirizzi fisici reali, dove i**

dati vengono effettivamente memorizzati nella RAM.

Se, ad esempio, vengono eseguiti tre processi contemporaneamente, ciascuno riceve un proprio **spazio di indirizzi virtuali** separato.
Ogni processo legge e scrive utilizzando questi indirizzi virtuali, senza sapere dove i dati siano effettivamente memorizzati nella RAM.

La **Memory Management Unit (MMU)** si occupa di:

- **tradurre gli indirizzi virtuali in indirizzi fisici, utilizzando una struttura chiamata tabella delle pagine.**
- Questo permette al sistema operativo di assegnare blocchi di memoria fisica in modo **non contiguo**, evitando sprechi e migliorando la gestione della RAM.

Ad Esempio:

Un processo potrebbe avere un intervallo di indirizzi virtuali che sembra continuo (`0x0000 - 0xFFFF`), ma i dati corrispondenti potrebbero essere distribuiti in diverse aree della RAM fisica (`0x12F3A000` , `0x3B12C800` , ecc.), in base alla disponibilità di memoria. Questo meccanismo consente una gestione più efficiente delle risorse e permette di eseguire processi anche se la RAM è frammentata.

System Call per la gestione della memoria

I processi possono richiedere memoria dinamicamente attraverso delle chiamate di sistema:

- `malloc` , `calloc` , `realloc` → **allocano dinamicamente blocchi di memoria.**
- `free` → **rilascia la memoria precedentemente allocata.**
- `brk` , `nmap` → **utilizzate dal sistema operativo per per gestire la memoria a livello più basso, spesso chiamate indirettamente tramite** `malloc` **.**

✓ Vantaggi dell'indirizzamento tramite mapping

Sicurezza: ogni processo viene isolato dagli altri, impedendo che un programma possa accedere o modificare accidentalmente (o intenzionalmente) i dati di un altro processo.
Flessibilità: consente di assegnare ai processi uno spazio di indirizzamento

virtuale maggiore rispetto alla memoria fisica disponibile. Ad esempio, un processo può utilizzare 4 GB di indirizzi anche se il sistema dispone solo di 2 GB di RAM.
Efficienza: permette al sistema operativo di gestire la memoria in modo più ottimizzato, allocando la RAM solo quando necessario e utilizzando tecniche come il *paging* (spostando parti della memoria virtuale su disco quando la RAM è piena).

Memoria Virtuale e Paging

La **memoria virtuale** è un sistema che permette al sistema operativo di **gestire la memoria in modo più efficiente**, **utilizzando sia la RAM fisica che una porzione del disco rigido chiamata swap space (o paging file), per simulare una quantità di memoria molto più ampia.**
Questo consente a ogni processo di avere accesso a uno **spazio di indirizzi virtuale privato**, che viene mappato sulla memoria fisica e sul disco in base alle necessità.

Gestione della memoria virtuale

- **Paging**:
  - **La memoria virtuale:**
    - **è suddivisa in blocchi chiamati pagine.**
  - **mentre nella memoria fisica:**
    - **i blocchi corrispondenti sono detti page frames.**
  - Non tutte le **pagine** di un processo vengono caricate in RAM contemporaneamente:
    - **alcune rimangono sul disco e vengono caricate solo quando necessario.**
- **Page Table**:
  Per tenere traccia della posizione delle pagine, il sistema operativo mantiene una **tabella delle pagine** per ciascun processo, che mappa le pagine virtuali ai frame delle pagine fisiche.
  Se la pagina non è presente nella memoria fisica (una condizione nota come **"[[#**Page Fault**|page fault]]"**), il sistema operativo la recupera dal disco e la carica in un frame di pagina libero nella memoria fisica, e aggiorna la tabella

per riflettere questo cambiamento.
Questa tabella mappa le pagine virtuali sui frame fisici della RAM e viene aggiornata quando le pagine vengono spostate tra RAM e disco.

- **Swap In / Swap Out**:
  Quando la RAM si riempie, il sistema operativo **sceglie le pagine meno utilizzate** e le sposta sul disco (**swapping out**) per liberare spazio. Quando una di queste pagine è nuovamente necessaria, viene ricaricata in RAM (**swapping in**).

Page Fault

Un **Page Fault** è un evento che:

- **si verifica quando un processo tenta di accedere a una pagina che non è presente nella RAM.**
- In questo caso, il sistema operativo deve recuperare la pagina dal disco e caricarla in un frame libero della RAM, aggiornando la tabella delle pagine.

Se la RAM è **troppo piena** e il sistema deve fare swap continuamente, si verifica un fenomeno chiamato **thrashing:**

- **che causa un calo drastico delle prestazioni perché il sistema passa più tempo a scambiare dati tra RAM e disco piuttosto che a eseguire i programmi.**

File system

Il file system è un livello di astrazione che il sistema operativo utilizza per gestire i dati sulla memoria secondaria **(hard disk, SSD, ecc.).**
Questo sistema è indipendente dal tipo e dal numero di dispositivi di archiviazione secondari presenti.
L'elemento base è il **file:**

> **cioè una sequenza di byte memorizzata su memoria secondaria.**

A differenza delle strutture dati in memoria, i file persistono anche dopo che il processo che li ha creati termina, spesso vengono terminati con il **EOF(end of file)**.

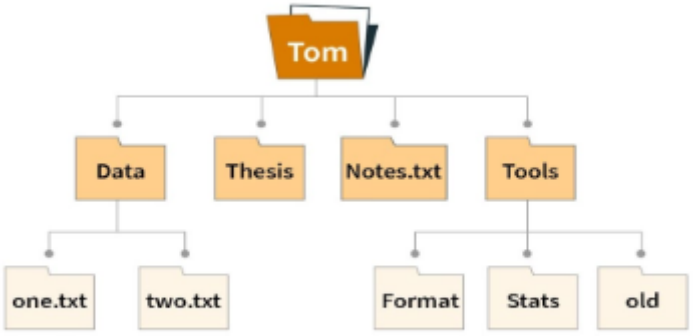
EOF (End of File)

Quindi un file può essere letto, scritto e cancellato.
Come detto poco sopra il processo che termina i file è l'EOF:
è una convenzione logica usata dal sistema operativo per capire quando un file termina.
Non si tratta di un simbolo fisico memorizzato all'interno del file (tranne in alcuni formati specifici), ma di una condizione gestita dal sistema tramite chiamate di sistema.
Il SO e le funzioni di lettura lo gestiscono in base alla lunghezza del file: quando non ci sono più dati da leggere, viene segnalato un **EOF**

> **Esempio pratico**
>
> Nei sistemi Windows l'EOF può essere rappresentato dal carattere speciale `CTRL+Z`
> Nei file binari e nei sistemi moderni, **non esiste un carattere fisico EOF** dentro il file.

Struttura delle directory

I file all'interno del file system sono organizzati in una struttura gerarchica di **directory e sottodirectory**.



File System Management.png

Anche il concetto di directory (o folder) è un'astrazione.

> **Più precisamente una directory è un file che contiene riferimenti (puntatori) ad altri file, stabilendo una relazione "genitore-figlio" tra il file system.**

I nomi dei file:

- **ovvero** il set di caratteri consentiti nei nomi dei file,
  e i metacaratteri utilizzati per:
- indicare la posizione dei file nel file system sono aspetti definiti da ciascun modello di file system specifico.

Esempio di percorsi di file

Windows:
"C:\Tom\Data\one.txt"
Questo è un percorso assoluto che identifica in modo univoco un file localizzato nel drive *C* in Microsoft Windows.

> **Nota**
>
> In Microsoft Windows, il percorso non è case-sensitive e specifica l'identificatore del drive fisico dove il file è archiviato.

Unix:
"~Tom/src/minimumSpanningTree.c"
Questo è un esempio tipico di percorso in un sistema UNIX (MacOS, Linux).

> **Nota**
>
> Questo percorso è case-sensitive, e nei sistemi Unix viene usata la convenzione "**~username**" per identificare la directory home degli utenti. Inoltre, il percorso è indipendente dalla posizione fisica del file su uno specifico dispositivo.

Operazioni sul file system

I processi possono interagire con il file system attraverso chiamate di sistema per:

- **Creare ed eliminare file**
- **Aprire e chiudere file (** `fopen` **,** `fclose` **)**
- **Leggere e scrivere dati (** `fread` **,** `fwrite` **)**
- **Modificare attributi (sola lettura, eseguibile, ecc.)**

Gestione dei dispositivi periferici

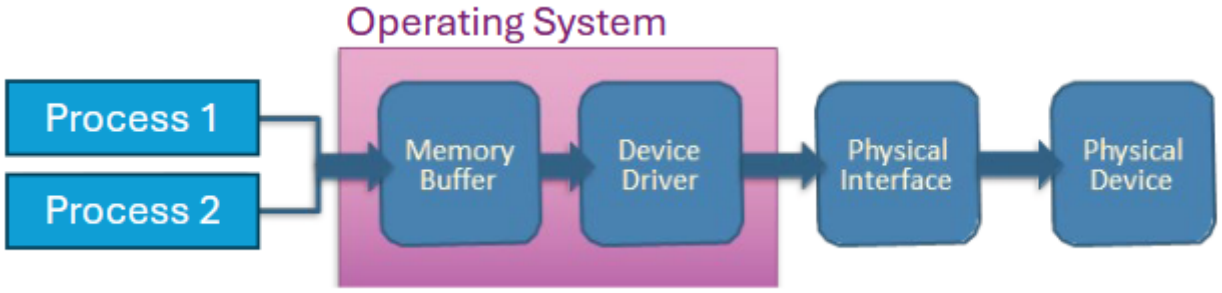
Il **sistema operativo** gestisce la comunicazione con le unità periferiche e fornisce un'astrazione (ovvero funzioni) che permette ai programmi di utilizzare il canale di comunicazione.

Poiché più programmi possono richiedere contemporaneamente l'accesso a un dispositivo specifico (ad esempio, output del terminale, input della tastiera o stampante), **il SO gestisce una coda di richieste per evitare conflitti (serializzazione).**

Per garantire una comunicazione efficiente con i dispositivi periferici, il SO utilizza un tipo speciale di memoria chiamata **memoria buffer:**

- **che funge da "area di sosta" dove i dati provenienti o destinati al dispositivo possono essere temporaneamente conservati fino al loro successivo trattamento.**

L'interazione con le periferiche avviene tramite un modulo software specifico, il **driver del dispositivo**.



Gestione dei dispositivi periferici.png

Le memoria secondarie

Le memorie secondarie sono i dispositivi di archiviazione persistenti: **possono conservare le informazioni registrate anche quando la macchina è spenta.**

Per ragioni fisiche e tecnologiche, le memorie secondarie hanno tempi di accesso e tassi di trasferimento più elevati rispetto alla memoria primaria (RAM) (la quale non ha componenti meccaniche).

Questi tipi di dispositivi (che possono essere chiamati anche **dispositivi di archiviazione di massa**) includono:

- **Hard Disk:**
  - HDD

    - SSD
    - SSD NMVe
- Schede di memoria
- Unità Flash USB (Pennette USB)
  E tutti i dispositivi hardware esterni che si connettono al computer tramite le porte USB, Thunderbolt, FireWire, PCI e PCIe.

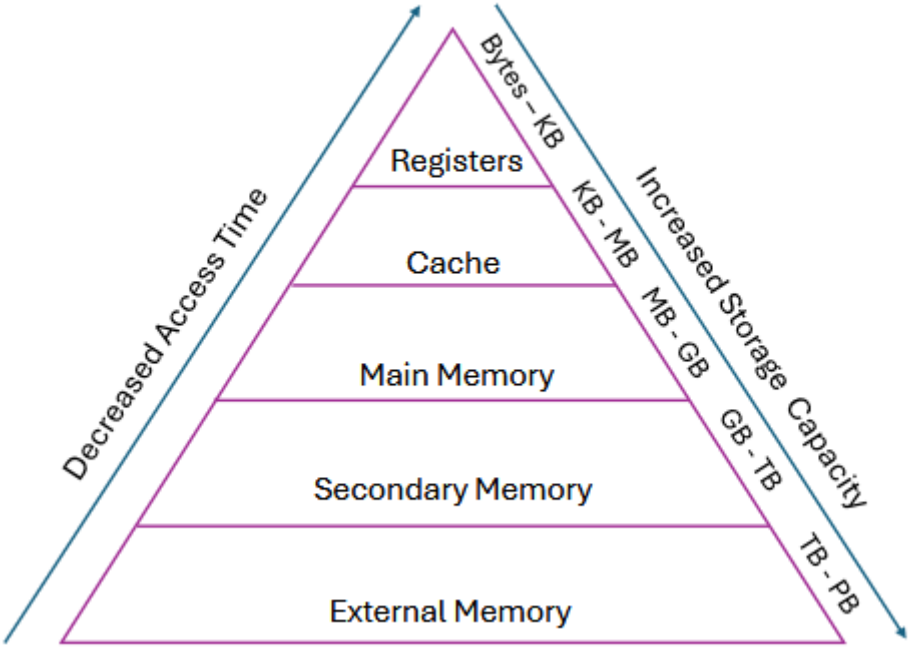
Le operazioni tipiche eseguite dal sistema operativo su questi dispositivi includono:

- **Allocazione e de-allocazione dello spazio per la memorizzazione dei dati (file)**
- **Gestione dello spazio libero all'interno dell'unità di memoria di massa**
- **Ottimizzazione, serializzazione e pianificazione delle operazioni all'interno dell'unità di memoria di massa**

La **gestione del file system** e la **gestione della memoria secondaria** sono due funzioni distinte, ma strettamente interconnesse, del sistema operativo.
Il modulo del sistema operativo responsabile della memoria secondaria ha il compito di:

- **nascondere la complessità fisica dei dispositivi di archiviazione, rendendoli trasparenti ai programmi.**
  In questo modo, **i file conservati su supporti differenti (come hard disk, SSD o unità rimovibili) possono essere gestiti attraverso le stesse chiamate di sistema, senza che i programmi debbano preoccuparsi delle caratteristiche hardware sottostanti**.

Gerarchie di memorie



center

Riprendendo quello che è stato detto poco sopra, questa immagine mostra come:

- a partire dalle memorie esterne fino ad arrivare ai registri della CPU, il tempo di accesso decresce,
- mentre, partendo dai registri della CPU fino ad arrivare alle memorie esterne la capacità di archiviazione cresce.

1. **Registri:**
   - **Memoria ultra veloce che conserva dati temporanei e informazioni cruciali per l'esecuzione delle istruzioni.**
   - Ad esempio:
     - IR, PC, etc.
2. **Cache:**
   - Memoria veloce che conserva i dati utilizzati più frequentemente per velocizzare l'accesso alle informazioni.
   - Ad esempio:
     - Temp
3. **Main Memory:**
   - **Memoria ad accesso rapido usata per eseguire processi e caricare dati, ma che non è persistente.**
   - Ad esempio:
     - RAM

4. **Memoria secondaria (Secondary Memory):**
   - **Memoria persistente usata per archiviare dati a lungo termine con accesso più lento.**
   - Ad esempio:
     - HHD, SSD
5. **Memoria esterna (External Memory):**
   - **Memorie esterne utilizzate per trasferire e archiviare dati a lungo termine con accesso via connessioni esterne.**
   - Ad esempio:
     - SSD, Chiavette USB, etc.

La gestione della protezione e della sicurezza

In un sistema multi-tasking e multi-utente, il sistema operativo gestisce la protezione delle risorse per garantire la privacy delle risorse e degli utenti.
Il processo di protezione si basa sui seguenti elementi:

1. **Autenticazione**:
   - **Una procedura per verificare l'identità dell'utente.**
2. **Autorizzazione**:
   - **Una procedura per determinare i diritti di un utente o di un processo nell'accedere a una risorsa.**

Autorizzazione

1. **Collegamento dei processi ai permessi degli utenti**:

- **Ogni processo nel sistema è associato a un utente specifico.**
- **I processi ereditano i permessi dell'utente associato per accedere alle risorse del sistema.**

2. Autorizzazione di alto livello per i processi del sistema operativo:
   - **I processi del sistema operativo vengono eseguiti dagli utenti con il livello di autorizzazione più elevato, come root o amministratore.**
3. **Politiche di sicurezza delle risorse:**
   - **Le politiche di sicurezza delle risorse si basano su regole che mappano i permessi di accesso alle risorse per gli utenti del**

sistema.

- Ad esempio:
  - i permessi di accesso ai file determinano quali utenti possono leggere, scrivere o eseguire file specifici.

4. **Raggruppamento degli utenti per semplificare l'autorizzazione:**
   - **Per semplificare la mappatura dell'autorizzazione, il sistema operativo consente di raggruppare gli utenti in cluster o gruppi.**
   - **Ogni membro all'interno di un gruppo eredita le autorizzazioni del gruppo**.

Autenticazione

Un sistema operativo multi-utente implementa procedure di login per l'accesso al sistema.
La procedura di login è cruciale per **2 motivi principali:**

1. **Autentica l'utente in base alle sue credenziali (ad esempio, nome utente e password).**
2. **Verifica se l'utente è autorizzato ad accedere al sistema.**

Il processo di login si basa su un archivio di credenziali utente e sulla definizione di gruppi di utenti, come:

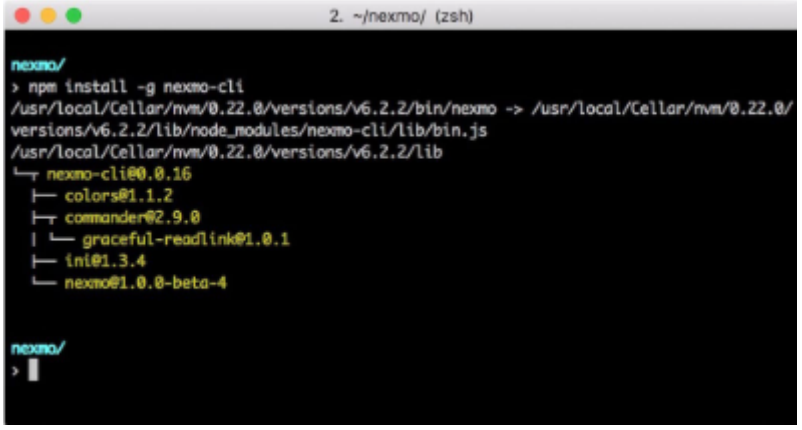
- **I file** `/etc/passwd`**,** `/etc/shadow` **e** `/etc/group/` **nei sistemi UNIX-like.**
- **Altri sistemi esterni di autenticazione e autorizzazione.**

> **NOTA**
>
> L'autenticazione basata su nome utente e password è la più comune, anche se non è l'unico o il metodo più sicuro.

Gestione del HCIs e delle applicazioni

Il sistema operativo fornisce un set di funzionalità per consentire alle applicazioni di creare strumenti di interazione con l'utente.
In generale, un'**interfaccia utente (UI)** può essere:
• **Interfaccia alfanumerica:**
Il sistema operativo offre un modello di terminale astratto per visualizzare

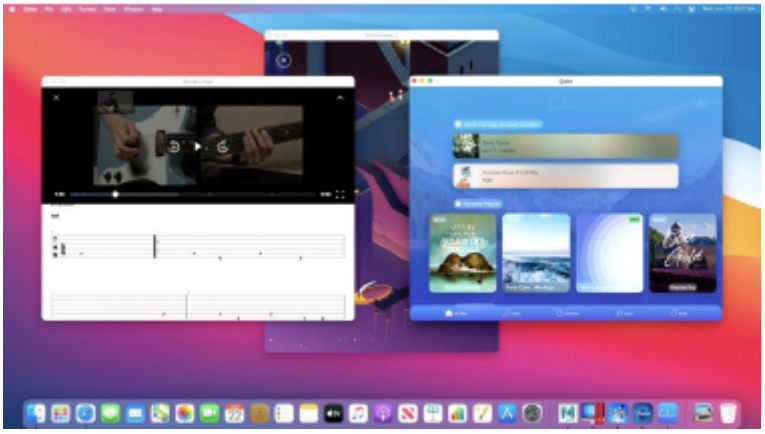
le informazioni su uno schermo che mostra caratteri alfabetici e numerici. Inoltre, cattura l'input tramite una tastiera.



Interfaccia Alfanumerica.png

- **Interfaccia grafica utente (GUI):**

Il sistema operativo fornisce funzionalità per permettere ai programmi di costruire una GUI utilizzando elementi come finestre e icone. L'input può essere effettuato utilizzando un mouse o uno schermo tattile, oltre alla tastiera.



GUI.png